

MAIN.PY

```

from fastapi import FastAPI, Depends
from dotenv import load_dotenv
from sqlmodel import create_engine, Session, SQLModel, select
from .models.User import User, UserRequest, UserResponse, UserResponseTres,
UserResponseSix, UserRequestSeven, UserRequestEight, UserRequestNine

import os
app = FastAPI()

load_dotenv()

DATABASE_URL= os.getenv("DATABASE_URL")
engine = create_engine(DATABASE_URL)

SQLModel.metadata.create_all(engine)

def get_db():
    db = Session(engine)
    try:
        yield db
    finally:
        db.close()

''' 1. CREATE - Afegir un nou registre a la taula'''
@app.post("/user", response_model=dict, tags=["CREATE"])
def addUser(user: UserRequest, db:Session = Depends(get_db)):
    #Convertir les dades del client a un model compatible amb la taula User
    insert_user = User.model_validate(user)
    db.add(insert_user)
    db.commit()
    return {"msg":"Afegit usuari correctament"}

''' 2. READ - Consutlar totes les dades d'un registre a la taula'''
@app.get("/user/{id}", response_model=UserResponse, tags=["READ by ID"])
def getUser(id: int, db:Session = Depends(get_db)):
    # Amb els statments de sqlalchemy es crea la consulta i es guarda a stmt
    stmt = select(User).where(User.id == id)
    result = db.exec(stmt).first()
    return UserResponse.model_validate(result)

''' 3. READ - Consultar totes les dades de tots els registres de la taula'''
@app.get("/api/user", response_model=list[UserResponseTres], tags=["READ ALL USERS"])
def getAllUsers(db:Session = Depends(get_db)):

```

```

#Dades sensibles: passwd i age
stmt = select(User)
allUsers = db.exec(stmt).all()
return allUsers

''' 4. READ - Consultar les dades filtrant per un camp '''
@app.get("/api/users/{lastname}", response_model=list[UserResponse], tags=["READ BY LASTNAME"])
def getUsersByLastname(lastname: str, db:Session = Depends(get_db)):
    stmt = select(User).where(User.lastname == lastname)
    result = db.exec(stmt).all()
    return result

''' 5. DELETE - Eliminar un registre per id '''
@app.delete("/api/delete_users/{id}", response_model=dict, tags=["DELETE BY ID"])
def deleteUserById(id:int, db:Session = Depends(get_db)):
    userById = db.get(User, id)
    db.delete(userById)
    db.commit()
    return {"msg":"Usuario borrado satisfactoriamente"}

''' 6. READ - Lectura parcial '''
@app.get("/api/users/get/partial/{id}", response_model=UserResponseSix, tags=["READ partial"])
def readUserPartialById(id:int, db:Session=Depends(get_db)):
    user_data = db.get(User, id)
    return UserResponseSix.model_validate(user_data)

''' 7. UPDATE - Modificació total (PUT)'''
@app.put("/api/users/updateAll/{id}", response_model=dict, tags=["UPDATE total"])
def updateAllUsers(id: int, user: UserRequestSeven, db:Session=Depends(get_db)):
    # Agafar les dades de l'usuari de la BD segons id
    user_db = db.get(User, id)
    # Convertir les dades del client en format dict
    user_data = user.model_dump()
    #Assignar valors del dict a l'usuari extret de la BD
    user_db.sqlmodel_update(user_data)
    #Enviar usuari amb dades modificades
    db.add(user_db)
    #Aplicar canvis a la taula
    db.commit()
    return {"msg":"Usuari modificat satisfactòriament"}

''' 8. UPDATE - Modificació parcial un camp (PATCH) '''
@app.patch("/api/users/updateOne/{id}", response_model=dict, tags=["UPDATE un camp"])
def updateOneField(id: int, user: UserRequestEight, db:Session = Depends(get_db)):
    user_db = db.get(User, id)

```

```
user_data = user.model_dump()

user_db.sqlmodel_update(user_data)

db.add(user_db)

db.commit()
return {"msg": "Usuari modificat satisfactòriament"}

''' 9. UPDATE - Modificació parcial dos camps '''
@app.patch("/api/users/updateTwo/{id}", response_model=dict, tags=["UPDATE dos
camps"])
def updateOneField(id: int, user: UserRequestNine, db: Session = Depends(get_db)):

    user_db = db.get(User, id)
    user_data = user.model_dump()

    user_db.sqlmodel_update(user_data)

    db.add(user_db)

    db.commit()
    return {"msg": "Usuari modificat satisfactòriament"}
```

/MODELS

Tot i que la meva solució és amb Users, serveix per a Product.

```
from sqlalchemy import SQLModel, Field

class User(SQLModel, table=True):
    id: int = Field(default=None, primary_key=True)
    name: str
    lastname: str
    age: int
    passwd: str #Sensible

class UserRequest(SQLModel):
    name: str
    lastname: str
    age: int
    passwd: str

class UserResponse(SQLModel):
    id: int
    name: str
    lastname: str
    age: int

class UserResponseTres(SQLModel):
    name: str
    lastname: str

''' En la següent classe es fa servir age i passwd com a dades sensibles'''
class UserResponseSix(SQLModel):
    id: int
    name: str
    lastname: str

class UserRequestSeven(SQLModel):
    name: str
    lastname: str
    age: int
    passwd: str

class UserRequestEight(SQLModel):
    age: int

class UserRequestNine(SQLModel):
    name: str
    age: int
```